Digital Pacemaker Project Report

Dr. Sudhanshu Choudhary

EECS 590

Justice J. Rebrovich

12/11/2025

Introduction

This project focuses on designing and simulating the core logic of a digital pacemaker using Verilog. The goal was to model how a pacemaker detects heartbeats; filters noise and generates a pacing pulse when the heart rate drops below a safe limit. All work was completed through Verilog modules and simulation.

System Overview

The design consisted of three main modules that work together to simulate pacemaker behavior.

1. ECG ROM- Produces a synthetic ECG waveform for simulation.
2. Adaptive LIF Detector- Heartbeat (QRS) detector
3. Pacemaker Controller- Determines when a pacing pulse should occur.

The ECG ROM outputs a repeating sequence of 12-bit values representing a simple heartbeat pattern. High values represent QRS spikes, mid values representing the baseline or no heartbeat at all. This provides a controlled input for evaluating the heartbeat detection logic.

The LIF system integrates the ECG sample, applies a dead zone to remove noise, and fires a one cycle pulse when the internal voltage v surpasses a threshold theta. It follows the pulse with a blanking and refractory period. The blanking period gives a small interval of no input and no sensing no matter what. The refractory period still senses new inputs but neglects them. This refractory period is represented by theta in the code and decreases exponentially after a pulse. The main purpose being to stop an instant fire from happening due to noise and smaller signals.

The pacemaker controller uses all the same logic I previously mentioned but is the determining factor for when to generate a pacing pulse. It also implements an escape interval giving a maximum allowed time between beats.

Results

Simulation confirmed that:

- QRS spikes from the ROM were successfully detected by the LIF module.
- The heart timing logic correctly reset after each detected beat.
- A pacing pulse was generated whenever no heartbeat occurred after the threshold.
- Blanking and refractory periods operated correctly to avoid false detections.

This demonstrates the functional behavior of a simplified pacemaker rhythm system.

Conclusion

The project successfully created and simulated a digital pacemaker model entirely on Verilog. While simplified, the design reflects key concepts used in real pacemaker systems, including sensing windows, time, and rate control. This was by far the most interesting and engaging project I have done. I can say I learned a lot and this class sparked an interest I did not know I had.

Code Reference

Project Summary | pacemaker.v

C:/VivadoProjects/Pacemaker Project Vivado File/Pacemaker Project/Pacemaker Project.srcs/sources_1/new/pacemaker.v

```
`timescale 1ns/1ps
module ecg_rom #( //electrocardiogram (electrical signal of heart)
parameter SAMPLE_COUNT = 10 //generates fake waveform
)(
    input wire clk,
    input wire rst,
    input wire [3:0] addr,
    output reg [11:0] data
);
always @(posedge clk) begin
    if (rst)
    data <= 12'd0;
    else begin
    case (addr)
        //first couple of beats
        4'd0: data <= 12'd4095; //4095 spike is normal heartbeat value(highest 12bit value)
        4'd1: data <= 12'd2048; //2048 is flatline, too many pacemaker will fire
        4'd2: data <= 12'd2048;
        4'd3: data <= 12'd2048;
        4'd4: data <= 12'd2048;
        4'd5: data <= 12'd2048;
        4'd6: data <= 12'd4095; //beats resume
        4'd7: data <= 12'd4095;
        4'd8: data <= 12'd4095;
        4'd9: data <= 12'd4095;
        default: data <= 12'd2048;
endcase
end
end
endmodule
```

```
    module adaptive_lif #( //leaky integrate-fire (neuron model to detect spikes)
        parameter SAMPLE_WIDTH = 12,
        parameter ACC_WIDTH = 20,
        parameter THETA_WIDTH = 16
)(
        input wire clk,
        input wire rst,
        input wire sample_clk_en,
        input wire signed [SAMPLE_WIDTH-1:0] sample_in,
        output reg spike_out,                       //1 tick pulse when beat detected
        output reg signed [ACC_WIDTH-1:0] v_out,
        output reg signed [THETA_WIDTH-1:0] theta_out //threshold
        );
        //Parameters
        localparam integer GAIN = 40; //sensitivity
        localparam integer DEAD_ZONE = 100; //noise threshold
        localparam integer V_RESET = 0;
        localparam integer THETA_BASE = 16'd1536;
        localparam integer THETA_INC = 16'd220; // increase threshold after spike(momentarily to prevent refire)
        localparam integer TAU_TH_DEC_SHIFT = 5;// decay to base rate
        localparam integer REFRACT_TICKS =3; // time it ingnores new inputs

        reg signed [ACC_WIDTH-1:0] v;
        reg signed [THETA_WIDTH-1:0] theta;
        reg [7:0] refract_cnt;
        integer signed_sample;
        integer drive;
always @(posedge clk) begin // spike or QRS detector functions
    if (rst) begin
    v <= 0;
    theta <= THETA_BASE;
    spike_out <= 0;
    refract_cnt <= 0;
    end else begin
        spike_out <= 0;
        if (sample_clk_en) begin
        if (refract_cnt > 0)
            refract_cnt <= refract_cnt -1;
        else begin
        signed_sample = sample_in - 12'sd2048;
        if ((signed_sample > DEAD_ZONE) || (signed_sample < -DEAD_ZONE))
            drive = signed_sample * GAIN;
        else
            drive = 0;
        v <= v + drive - (v >>> 4);
        if (v >= theta) begin
        spike_out <= 1;
        v <= V_RESET;
        theta <= theta + THETA_INC;
        refract_cnt <= REFRACT_TICKS;
        end else begin
            theta <= theta - ((theta - THETA_BASE) >>> TAU_TH_DEC_SHIFT);
        end
    end
  end
  v_out <= v;
  theta_out <= theta;
  end
  end
  endmodule
module pacemaker_ctrl #(
    parameter integer SAMPLE_FREQ = 1000,
    parameter integer LOWER_RATE_BPM = 60,
    parameter integer BLANKING_MS = 40,
    parameter integer REFRACT_MS = 200
)(
    input wire clk,
    input wire rst,
    input wire sample_clk_en,
    input wire detector_spike,
    output reg pace_pulse
);
    localparam integer ESCAPE_INTERVAL_TICKS = (60 * SAMPLE_FREQ) / LOWER_RATE_BPM;
    localparam integer BLANK_TICKS = (BLANKING_MS * SAMPLE_FREQ) / 1000;
    localparam integer REFRACT_TICKS = (REFRACT_MS * SAMPLE_FREQ) / 1000;

        integer timer_since_event;
        integer blank_until;
        integer refract_until;
        integer tick;
```

```
111
112 always @(posedge clk) begin
113     if (rst) begin
114     timer_since_event <= ESCAPE_INTERVAL_TICKS;
115     blank_until <= 0;
116     refract_until <= 0;
117     tick <= 0;
118     pace_pulse <= 0;
119 end else begin
120     pace_pulse <= 0;
121     if (sample_clk_en) begin
122     tick <= tick + 1;
123     if (detector_spike && tick >= blank_until && tick >= refract_until) begin
124         timer_since_event <= 0;
125         refract_until <= tick + REFRACT_TICKS;
126     end else begin
127         timer_since_event <= timer_since_event + 1;
128     end
129         if (timer_since_event >= ESCAPE_INTERVAL_TICKS) begin
130             pace_pulse <= 1;
131             timer_since_event <= 0;
132             blank_until <= tick + BLANK_TICKS;
133 end
134 end
135 end
136 end
137 endmodule
138
139 module top (
140     input wire CLK,
141     input wire RSTn,
142     output wire LED_HEART,// natural beat
143     output wire LED_PACE // pace pulse
144 );
145     parameter integer BOARD_CLK_HZ = 100_000_000; // reduce speed of fpga clk
146     parameter integer SAMPLE_RATE = 1000;
147     parameter integer SAMPLE_COUNT = 10;
148     localparam integer DIV = BOARD_CLK_HZ / SAMPLE_RATE;
149
150     reg[31:0] divcnt = 0;
151     reg sample_clk_en = 0;

150     reg[31:0] divcnt = 0;
151     reg sample_clk_en = 0;
152
153     always @(posedge CLK) begin
154         if (!RSTn) begin
155         divcnt <= 0;
156         sample_clk_en <= 0;
157     end else begin
158         if (divcnt >= DIV -1) begin
159         divcnt <= 0;
160         sample_clk_en <= 1;
161     end else begin
162         divcnt <= divcnt +1;
163         sample_clk_en <= 0;
164 end
165 end
166 end
167
168         reg [3:0] sample_addr = 0;
169         always @(posedge CLK) begin
170             if (!RSTn)
171                 sample_addr <= 0;
172             else if (sample_clk_en)
173                 sample_addr <= (sample_addr == SAMPLE_COUNT-1) ? 0 : sample_addr +1;
174 end
175     wire [11:0] ecg_sample;
176     ecg_rom rom_inst (
177     .clk(CLK) ,
178     .rst(~RSTn) ,
179     .addr(sample_addr) ,
180     .data(ecg_sample)
181 );
182     wire spike;
183     wire signed [19:0] v_out;
184     wire signed [15:0] theta_out;
---
185
186     adaptive_lif lif_inst (
187     .clk(CLK) ,
188     .rst(~RSTn) ,
189     .sample_clk_en(sample_clk_en) ,
190     .sample_in(ecg_sample) ,
191     .spike_out(spike) ,
192     .v_out(v_out) ,
193     .theta_out(theta_out)
194 );
195
196 wire pace;
197     pacemaker_ctrl pm_inst (
198         .clk(CLK) ,
199         .rst(~RSTn) ,
200         .sample_clk_en(sample_clk_en) ,
201         .detector_spike(spike) ,
202         .pace_pulse(pace)
203 );
204     assign LED_HEART = spike;
205     assign LED_PACE = pace;
206 endmodule
207
```

Testbench & Schematic

```
`timescale 1ns / 1ps
    module testbench;

        reg CLK;
        reg RSTn;
        wire LED_HEART;
        wire LED_PACE;

    initial CLK=0;
    always #5 CLK = ~CLK; //clk generator
        initial begin
        RSTn = 0; // reset generator
        #100;
        RSTn =1;
    end
top dut ( //top module as device under test
    .CLK(CLK) ,
    .RSTn(RSTn) ,
    .LED_HEART(LED_HEART) ,
    .LED_PACE(LED_PACE)
);
    initial begin
        $display("Time(ns) LED_HEART LED_PACE"); //graph label
        $monitor("%0t %b %b", $time, LED_HEART, LED_PACE); //shows data under header label
        #(200_000_000); //200ms
        $finish;
end
endmodule
```

